

Reviewer Pack — Free Probability Tensor Product over DPLL Proof Length

Entry 5acea8240273 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 21:24:48 UTC

Free Probability Tensor Product over DPLL Proof Length

Entry ID: 5acea8240273 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 21:24:48 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Complexity Theory: DPLL Proof Complexity

Statement:

For a given Boolean formula F , the minimal noncommutative information entropy of its associated density matrix $M(F)$ in free probability theory is upper-bounded by $\log(n!)^{(1/2)}$, where n is the number of variables in F , and this bound is tight for formulas with DPLL proof length proportional to $\log(n!)$.

Rationale (proposer's reasoning):

Free probability theory provides a noncommutative generalization of classical probability theory, which has been used to study entanglement in quantum systems. By applying free probability to Boolean formulas and relating it to the DPLL proof complexity, this conjecture seeks to expose hidden structures in computational complexity through a novel mathematical lens.

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): aed4cb78aec5a914

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if all 30 seeds produce density matrices $M(F)$ with noncommutative information entropy $H(M(F)) \leq \log(n!)^{(1/2)}$ for formulas F with DPLL proof length $L(F)$ proportional to $\log(n!)$. The conjecture is falsified if any seed produces a formula F with $H(M(F)) > \log(n!)^{(1/2)}$ and a DPLL proof length $L(F)$ not scaling proportionally to $\log(n!)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free probability" AND "DPLL proof complexity" - "density matrix" free probability AND DPLL proof - "noncommutative information entropy" free probability AND DPLL proof length

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_3cnf(n, m):
    clauses = []
    for _ in range(m):
        clause = set()
        while len(clause) < 3:
            lit = random.randint(-n, n)
            if -lit not in clause and lit != 0:
                clause.add(lit)
        clauses.append(tuple(sorted(clause)))
    return clauses

def free_probability_tensor_product(clauses):
    n = max(abs(lit) for clause in clauses for lit in clause)
    M = [[0] * (2**n) for _ in range(2**n)]

    def add_clause(clause):
        for i in range(1 << n):
            for j in range(1 << n):
                if all((i >> k & 1) ^ (j >> k & 1) == int(lit > 0) for lit in clause):
                    M[i][j] += 1

    for clause in clauses:
        add_clause(clause)

    # Normalize the matrix
    total = sum(sum(row) for row in M)
    for i in range(2**n):
        for j in range(2**n):
            M[i][j] /= total
```

```

    return M

def noncommutative_information_entropy(M, n):
    entropy = 0.0
    for i in range(1 << n):
        for j in range(1 << n):
            if M[i][j] > 0:
                entropy -= M[i][j] * math.log2(M[i][j])
    return entropy

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    m = random.randint(n**2, n**3)
    clauses = generate_random_3cnf(n, m)

    M = free_probability_tensor_product(clauses)
    entropy = noncommutative_information_entropy(M, n)

    max_log_fact_n = math.log(math.factorial(n)) ** 0.5
    conjecture_holds = entropy <= max_log_fact_n

    return {
        "metric_name": "noncommutative_information_entropy",
        "metric_value": entropy,
        "instances_tested": len(clauses),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"Entropy {entropy} > {max_log_fact_n}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_58a3e2a5.py", line 85, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_58a3e2a5.py", line 65, in run_trial
    M = free_probability_tensor_product(clauses)
        ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_58a3e2a5.py", line 31, in free_probabil
    M = [[0] * (2**n) for _ in range(2**n)]
          ~~~~~
```

MemoryError

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a MemoryError before producing data that could confirm or refute the conjecture. | next: Increase the memory allocation for the test or optimize the code to handle larger inputs and then retest the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10896
2	preregistration	ollama_remote	glm4:latest	0	0	6083
3	novelty	ollama_remote	glm4:latest	0	0	4503
4	novelty	ollama_remote	glm4:latest	0	0	6208
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15085
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10285
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8716
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9721
9	judge	ollama_remote	glm4:latest	0	0	7939

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 79436 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5acea8240273.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5acea8240273.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5acea8240273.tar.gz` (if generated)